Artificial Intelligence

AI-2002

Lab 05

Informed Search Algorithms

National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2002	Artificial Intelligence Lab

1. Objective 3

2. Informed Search 3

2.1 Best First search 4

2.2 Greedy Best First Search 6

2.3 A* Search 8

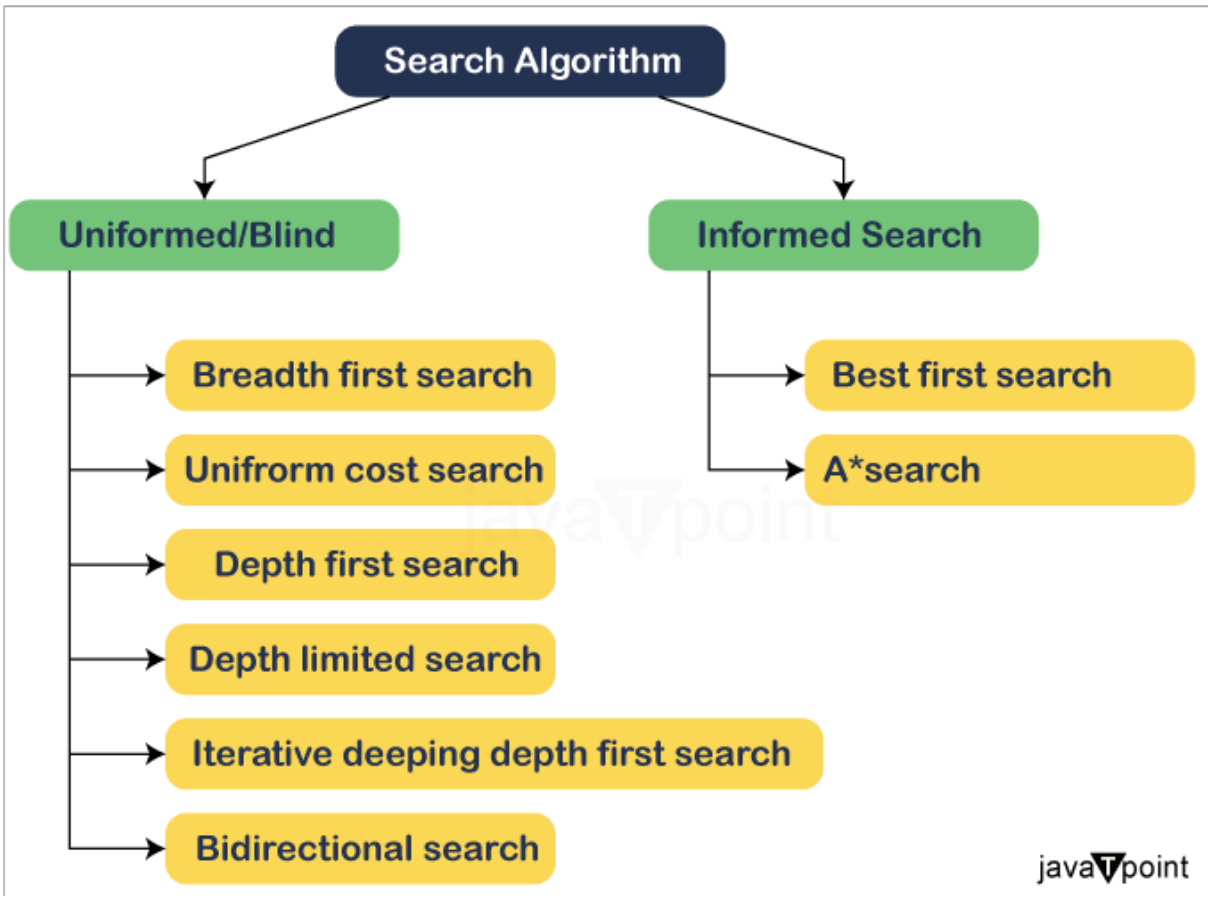
3. Task 10

1. Objective

1. Introduction to Problem Solving by Searching
2. Implementing Informed/Heuristic Search Algorithms in Python

2. Heuristic (or informed) search algorithms:

Informed search in AI refers to a category of search algorithms that utilize problem-specific knowledge beyond the definition of the problem itself to find solutions more efficiently than uninformed search algorithms. These algorithms leverage additional information, often in the form of heuristics, to guide the search process toward the most promising paths, thereby reducing the search space and improving performance.

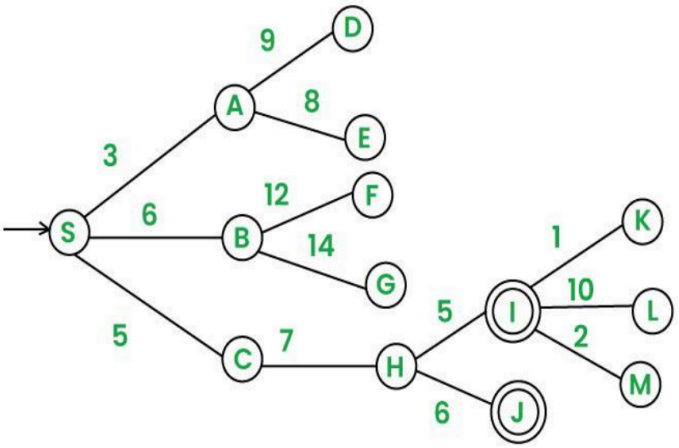


2.1 Best-First Search

If we consider searching as a form of traversal in a graph, an uninformed search algorithm would blindly traverse to the next node in a given manner without considering the cost associated with that step. An informed search, like Best first search, on the other hand would use an evaluation function to decide which among the various available nodes is the most promising (or 'BEST') before traversing to that node.
The Best first search uses the concept of a **Priority queue** and heuristic search. To search the graph space, the BFS method uses two lists for tracking the traversal. An 'Open' list which keeps track of the current 'immediate' nodes available for traversal and 'CLOSED' list that keeps track of the nodes already traversed.

Graph Structure: The graph is represented as a dictionary, where each key is a parent node, and the value is a list of its children and the heuristic. For example:

```
graph = {
    'S': [('A', 3), ('B', 6), ('C', 5)],
    'A': [('D', 9), ('E', 8)],
    'B': [('F', 12),
('G', 14)],
    'C': [('H', 7)],
    'H': [('I', 5),
('J', 6)],
    'I': [('K', 1),
('L', 10), ('M', 2)],
    'D': [],'E': [],
    'F': [],'G': [],
    'J': [],'K': [],
    'L': [],'M': []
}
```



BFS Function: bfs(graph, start, goal)
The bfs function performs a best first search on the given tree (graph) starting from the start node and searches for the goal node.

- **Parameters:**
    1. graph: The tree to be traversed.

2. start: The node where the search begins.
3. goal: The node that the algorithm is trying to find.

- **Process:**
  1. Create 2 empty lists: OPEN and CLOSED.
  2. Start from the start node and put it in the 'ordered' OPEN list.
  3. The search continues as long as there are nodes in the queue:
     - If OPEN list is empty, then EXIT the loop returning 'False'
     - Select the first/top node (say N) in the OPEN list and move it to the CLOSED list. Also capture the information of the parent node.
     - If N is a GOAL node, then move the node to the Closed list and exit the loop returning 'True'. The solution can be found by backtracking the path.
     - If N is not the GOAL node, expand node N to generate the 'immediate' next nodes linked to node N and add all those to the OPEN list
     - Reorder the nodes in the OPEN list in ascending order according to an evaluation function f(n)

```
def best_first_search(graph, start, goal):
    visited = set()
    pq = PriorityQueue()
    pq.put((0, start))  # priority queue with priority as the heuristic
value
    while not pq.empty():
        cost, node = pq.get()
        if node not in visited:
            print(node, end=' ')
            visited.add(node)
            if node == goal:
                print("\nGoal reached!")
                return True
            for neighbor, weight in graph[node]:
                if neighbor not in visited:
                    pq.put((weight, neighbor))
    print("\nGoal not reachable!")
    return False
```

```
 # Example usage:
print("Best-First Search Path:")
best_first_search(graph, 'A', 'I')
```

Maze Game using Best First Search

We apply Best-First Search to solve a maze. The algorithm uses the Manhattan distance heuristic to guide the search towards the goal. Each node in the maze is represented by its position and its cost estimates.

```
from queue import PriorityQueue
class Node:
    def __init__(self, position, parent=None):
        self.position = position
        self.parent = parent
        self.g = 0  # cost from start node to current node
        self.h = 0  # heuristic estimate of the cost from current node
to end node
        self.f = 0  # total cost

    def __lt__(self, other):
        return self.f < other.f

def heuristic(current_pos, end_pos):
    # Manhattan distance heuristic
    return abs(current_pos[0] - end_pos[0]) + abs(current_pos[1] -
end_pos[1])

def best_first_search(maze, start, end):
    rows, cols = len(maze), len(maze[0])
    start_node = Node(start)
    end_node = Node(end)
    frontier = PriorityQueue()
    frontier.put(start_node)
    visited = set()
```

```
    while not frontier.empty():
        current_node = frontier.get()
        current_pos = current_node.position
        if current_pos == end:
            path = []
            while current_node:
                path.append(current_node.position)
                current_node = current_node.parent
            return path[::-1]  # Reverse the path to start from the
start position
        visited.add(current_pos)
        # Generate adjacent nodes
        for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            new_pos = (current_pos[0] + dx, current_pos[1] + dy)
            if 0 <= new_pos[0] < rows and 0 <= new_pos[1] < cols and
maze[new_pos[0]][new_pos[1]] == 0 and new_pos not in visited:
                new_node = Node(new_pos, current_node)
                new_node.g = current_node.g + 1
                new_node.h = heuristic(new_pos, end)
                new_node.f = new_node.h  # Best-First Search: f(n) =
h(n)
                frontier.put(new_node)
                visited.add(new_pos)
    return None  # No path found

# Example maze
maze = [
    [0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0],
    [0, 0, 1, 0, 1],
    [0, 0, 1, 0, 0],
    [0, 0, 0, 1, 0]
]
start = (0, 0)
end = (4, 4)

path = best_first_search(maze, start, end)
if path:
    print("Path found:", path)
```

```
else:
    print("No path found")
```

2.2 Greedy Best First Search Algorithm

A search method of selecting the best local choice at each step in hopes of finding an optimal solution. It is the combination of depth-first search and breadth-first search algorithms. At each step, we choose the most promising node. In the greedy search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e. f(n)= h(n).

```
# Graph with different edge costs
graph = {
    'A': {'B': 2, 'C': 1},
    'B': {'D': 4, 'E': 3},
    'C': {'F': 1, 'G': 5},
    'D': {'H': 2},
    'E': {},
    'F': {'I': 6},
    'G': {},
    'H': {},
    'I': {}
}
# Heuristic function (estimated cost to reach goal 'I')
heuristic = {'A': 7,'B': 6,'C': 5,'D': 4,'E': 7,'F': 3,'G': 6,'H': 2,'I': 0}

# Greedy Best-First Search Function (without heapq)
def greedy_bfs(graph, start, goal):
    frontier = [(start, heuristic[start])]  # List-based priority queue
(sorted manually)
    visited = set()  # Set to keep track of visited nodes
    came_from = {start: None}  # Path reconstruction

    while frontier:
        # Sort frontier manually by heuristic value (ascending order)
        frontier.sort(key=lambda x: x[1])
```

```
        current_node, _ = frontier.pop(0)  # Get node with best heuristic
        if current_node in visited:
            continue

        print(current_node, end=" ")  # Print visited node
        visited.add(current_node)

        # If goal is reached, reconstruct path
        if current_node == goal:
            path = []
            while current_node is not None:
                path.append(current_node)
                current_node = came_from[current_node]
            path.reverse()
            print(f"\nGoal found with GBFS. Path: {path}")
            return

        # Expand neighbors based on heuristic
        for neighbor in graph[current_node]:
            if neighbor not in visited:
                came_from[neighbor] = current_node
                frontier.append((neighbor, heuristic[neighbor]))
    print("\nGoal not found")
# Run Greedy Best-First Search
print("\nFollowing is the Greedy Best-First Search (GBFS):")
greedy_bfs(graph, 'A', 'I')
```

Disadvantage – It can get stuck in loops. It is not optimal.

Difference between Best First Search & Greedy Best First Search:
BFS uses an evaluation function f(n) to prioritize nodes, which **can include factors like cost g(n)) or heuristic estimates h(n))**. It is flexible but not guaranteed to be complete or optimal unless the evaluation function is carefully designed. Its behavior depends on how f(n) is defined. A specific variant of BFS **(Greedy BFS) uses only the heuristic function h(n)** to greedily select the node

closest to the goal. It ignores the cost to reach the node g(n)), making it fast but not guaranteed to be complete or optimal. It can produce suboptimal solutions or get stuck in loops.

2.3 A* search (A- Star Search)

The A* algorithm is a widely used informed search algorithm that combines the strengths of uniform-cost search (using g(n), the cost to reach the current node) and greedy best-first search (using h(n), the heuristic estimate of the cost to the goal).

Evaluation function f(n) = g(n) + h(n)

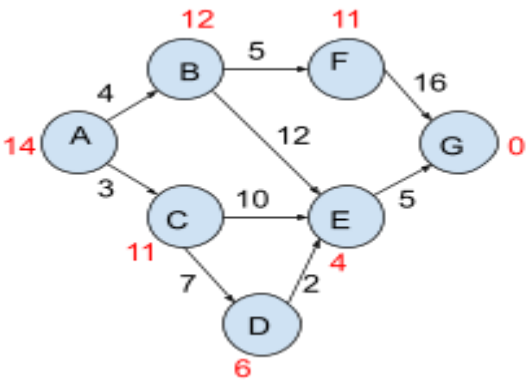
g(n) = cost **so far** to reach n
h(n)= estimated cost from n to goal

Graph Structure: The graph is represented as a dictionary, where each key is a parent node, and the value is again a dictionary which contains neighbours of the key and the cost required to reach that key.

Heuristics for each node are given separately in the form of a dictionary. For example:

```
from queue import PriorityQueue
# Graph with different edge costs
graph = {
    'A': {'B': 4, 'C': 3},
    'B': {'E': 12, 'F': 5},
    'C': {'D': 7, 'E': 10},
    'D': {'E': 2},
    'E': {'G': 5},
    'F': {'G': 16},
    'G': {},
}

heuristic = {'A': 14,'B': 12,'C': 11,'D': 6,'E': 4,'F': 11,'G': 0 }
```



A* Function: a_star(graph, start, goal)
The function performs A* search on the given tree (graph) starting from the start node and search for the goal node.

- **Parameters:**
  1. graph: The tree to be traversed.
  2. start: The node where the search begins.
  3. goal: The node that the algorithm is trying to find.
- **Process:**
  1. Initialize two lists: Open List (nodes to be evaluated, starting with the initial node) and Closed List (nodes already evaluated, initially empty).
  2. Assign values to the initial node: g(n)=0, $h(n)$ (heuristic estimate to the goal), and $f(n)=g(n)+h(n)$.
  3. Add the initial node to the Open List.
  4. Select the node with the lowest $f(n)$ from the Open List.
  5. Remove this node from the Open List and add it to the Closed List.
  6. Check if the selected node is the goal. If yes, reconstruct the path by backtracking using parent pointers.
  7. If the goal is not found, expand the selected node by generating all its successors.
  8. For each successor:
     - Calculate $g(n)$ as g(current node)+cost to move to the successor.
     - Calculate $h(n)$ (heuristic estimate to the goal).
     - Calculate $f(n)=g(n)+h(n)$.
  9. If the successor is in the Closed List and the new $g(n)$ is higher, skip it.
  10. If the successor is in the Open List and the new $g(n)$ is lower, update its $g(n)$, $h(n)$, and $f(n)$, and set its parent to the current node.
  11. If the successor is not in either list, add it to the Open List and set its parent to the current node.
  12. Repeat the process until the goal is added to the Closed List or the Open List is empty.
  13. If the goal is found, backtrack from the goal to the start node using parent pointers to reconstruct the optimal path.
  14. If the Open List is empty and the goal is not found, no solution exists.

```
def a_star(graph, start, goal):
    frontier = [(start, 0 + heuristic[start])]  # List-based priority
```

```
queue (sorted manually)
    visited = set()  # Set to keep track of visited nodes
    g_costs = {start: 0}  # Cost to reach each node from start
    came_from = {start: None}  # Path reconstruction

    while frontier:
        # Sort frontier by f(n) = g(n) + h(n)
        frontier.sort(key=lambda x: x[1])
        current_node, current_f = frontier.pop(0)  # Get node with
lowest f(n)

        if current_node in visited:
            continue

        print(current_node, end=" ")  # Print visited node
        visited.add(current_node)

        # If goal is reached, reconstruct path
        if current_node == goal:
            path = []
            while current_node is not None:
                path.append(current_node)
                current_node = came_from[current_node]
            path.reverse()
            print(f"\nGoal found with A*. Path: {path}")
            return

        # Explore neighbors
        for neighbor, cost in graph[current_node].items():
            new_g_cost = g_costs[current_node] + cost  # Path cost from
start to neighbor
            f_cost = new_g_cost + heuristic[neighbor]  # f(n) = g(n) +
h(n)

            if neighbor not in g_costs or new_g_cost <
g_costs[neighbor]:
                g_costs[neighbor] = new_g_cost
                came_from[neighbor] = current_node
                frontier.append((neighbor, f_cost))
```

```
    print("\nGoal not found")

# Run A* Search
print("\nFollowing is the A* Search:")
a_star(graph, 'A', 'G')
```

LAB TASKS

TASK #1

A search-and-rescue drone needs to navigate a disaster zone represented as a grid, where different terrain types (e.g., "Road" = cost 1, "Rubble" = cost 3, "Water" = cost 5, "Blocked" = impassable) affect how much energy the drone consumes to cross each cell, rather than every move costing a uniform 1 unit. Implement A* Search on a grid of at least 6x6 cells with mixed terrain types, using the Manhattan distance as the heuristic, but where g(n) must now account for the actual terrain cost of each step rather than a flat step count. The drone must find the least-energy path from a starting cell to a survivor's location, and the program must print the total energy cost of the optimal path alongside the path itself. As an added challenge, verify that your heuristic remains admissible even with variable terrain costs (hint: the heuristic should assume the cheapest possible terrain type when estimating remaining distance).

TASK #2

Build a graph of at least 10 nodes with edge costs and heuristic values, designed so that Greedy Best-First Search gets lured toward a node with a low heuristic but a high actual edge cost, causing it to pick a worse path than A*. Run both Greedy BFS and A* on this graph for the same start/goal pair, print the path and total cost found by each, and print a

TASK #3

Implement A* Search on a weighted graph of at least 9 cities, but change the evaluation function to f(n) = g(n) + w * h(n), where w is adjustable. Run the search with at least 4 different values of w (e.g., 0.5, 1.0, 1.5, 2.0), and for each, print the path, total cost, and number of nodes expanded. Briefly note in the output how higher w values push the algorithm toward Greedy behavior while lower values push it toward Uniform Cost behavior.